Qiskit Fall Fest 2026

Phase 1 — Quantum Processor & Architecture Starter Notebook

Geometry-aware quantum computing • Processor A/B benchmarking

---

Participant Guide

Goal: Learn the concepts needed to understand the challenge, build your selected Problem Statement solution, run it on **Processor A and Processor B**, and produce a reproducible architecture-aware comparison.

Notebook roadmap

Section	Topic
1	How to Use This Notebook
2	Phase 1 Challenge Overview
3	Repository Structure
4	Core Mental Model
5	Quantum Foundations
6	Processor as a Graph
7	Bell-State Example
8	Routing & SWAP Overhead
9	Transpilation
10	Noise
11	Required Measurements
12	Processor Definitions
13	Problem Statement
14	A/B Experiment
15	A/B Comparison
16	Protection / Error Detection
17	Scientific Reporting
18	Results & Figures
19	Experience-Level Approaches
20	Workflow Checklist
21	Final Experiment Structure
22	Submission Checklist
23	Participant Lab Notes

> ***Important:*** *This notebook is* ***Phase 1 only****. It focuses exclusively on the A/B benchmark required in the online qualification stage. The official Starter Guide remains authoritative for processor definitions, PS-specific inputs, metrics and benchmark constraints.*

1. How to Use This Notebook

This notebook is designed to be followed **top to bottom**.

Recommended workflow

1. Read the challenge overview.
2. Create the repository structure.
3. Learn the processor/architecture concepts.
4. Select **one** Problem Statement.
5. Implement the PS solution.
6. Prepare the A/B benchmark.
7. Run the same solution on Processor A.
8. Run the same solution on Processor B.
9. Record problem and architecture metrics.
10. Compare A vs B using tables and figures.
11. Explain the observed differences.
12. Complete the reproducibility and submission checklist.

Three rules

Rule 1 — One PS: Your Phase-1 submission must clearly identify exactly one selected Problem Statement.

Rule 2 — Controlled comparison: Keep the benchmark conditions consistent between Processor A and Processor B.

Rule 3 — Evidence first: Explain architectural differences using measured results, not unsupported assumptions.

2. Phase 1 Challenge Overview

2.1 The Phase-1 objective

The official Phase 1 instructions summarize the task as:

Choose one PS → build the solution → reproduce it on Processor A and Processor B → analyze how the two architectures affect the solution → submit a structured, reproducible repository.

2.2 What you are expected to compare

For each processor, you should record:

- the PS-specific performance metric(s),
- relevant circuit/resource metrics,
- the effect of architecture on execution,
- the evidence supporting your explanation.

The official instructions give examples such as energy error, objective value, accuracy, fidelity, classification performance and benchmark time, together with architecture/resource measures such as depth, two-qubit gates, SWAPs, qubits, execution cost or runtime.

2.3 The central question

> ***How does the processor architecture change the way the same quantum solution executes?***

That question should guide the entire notebook.

3. Recommended Repository Structure

```
project/
├── README.md
├── main.ipynb
├── requirements.txt
├── data/
│   ├── raw/
│   └── processed/
├── processors/
│   ├── processor_A.json
│   └── processor_B.json
├── src/
│   ├── __init__.py
│   ├── problem.py
│   ├── processors.py
│   ├── routing.py
│   ├── metrics.py
│   └── visualization.py
├── results/
│   ├── tables/
│   └── figures/
├── report.md
└── assets/
    └── qff_phase1_latex_images/
        ├── eq_qubit_state.png
        ├── eq_bell_state.png
        ├── eq_graph.png
        ├── eq_bell_fidelity.png
        ├── eq_zz_expectation.png
        └── eq_standard_error.png
```

File-by-file purpose

Path	What participants should put there
`README.md`	Selected PS, setup, execution, methodology, key results
`main.ipynb`	Reproducible end-to-end experiment
`requirements.txt`	Required software dependencies
`data/`	PS-specific input/processed data
`processors/`	Official Processor A/B definitions/configuration
`src/`	Reusable implementation modules
`results/tables/`	CSV/JSON result tables
`results/figures/`	Architecture, circuit and performance figures
`report.md`	Analysis, interpretation, limitations and conclusions

The official Phase 1 submission package requires the core implementation, processor definitions/configuration, results, README, analysis/report, environment specification and supporting figures.

> ***Rendering note:*** *The mathematical expressions used in this notebook are provided as PNG assets so they remain visible even in notebook viewers with limited LaTeX/MathJax support.*

4. The Core Mental Model

The central idea is:

> ***Your algorithm describes the logical operations. The processor architecture determines how those operations can actually be executed.***

```
Problem / Algorithm
        ↓
Logical quantum circuit
        ↓
Placement + routing + gate decomposition
        ↓
Physical processor
(qubits + coupling graph + gates + noise)
        ↓
Measurements
        ↓
Problem metric + architecture metrics
```

A mathematically identical logical circuit can produce different compiled circuits on different processor architectures.

That is why this hackathon asks you to compare the architecture, not just the final answer.

5. Quantum Foundations You Need

5.1 Qubit

`|ψ⟩ = α|0⟩ + β|1⟩, |α|² + |β|² = 1.`

5.2 Important Gates

- `X` — bit flip
- `H` — creates superposition
- `Z` — phase flip
- `RX`, `RY`, `RZ` — rotations
- `CX/CNOT` — two-qubit controlled operation

Example:

```
q0: ── H ──●──
           |
q1: ───────X──
```

This creates the Bell state

`|ψ⟩ = α|0⟩ + β|1⟩, |α|² + |β|² = 1`

Two-qubit gates matter heavily because they depend on processor connectivity.

6. A Quantum Processor as a Graph

Represent a processor as

`G = (V, E)`

where:

- `V` = physical qubits
- `E` = allowed physical two-qubit interactions.

Example:

```
0 — 1 — 2 — 3 — 4
```

means that `0↔1`, `1↔2`, `2↔3`, `3↔4` are available, but `0↔4` is not a direct interaction.

6.1 Why Routing Exists

If the logical circuit asks for a two-qubit operation between qubits that are not adjacent in the physical graph, the compiler has to transform the circuit, often by inserting SWAPs or equivalent routing operations.

That can increase:

- circuit depth,
- two-qubit gate count,
- SWAP count,
- physical resource usage,
- exposure to noise.

The challenge brief explicitly identifies routing as a source of extra operations whose effect can become important under noise.

```
import json, math
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import networkx as nx

processor_toy = {
    "name": "toy_5q_line",
    "num_qubits": 5,
    "coupling_map": [[0,1],[1,2],[2,3],[3,4]],
    "basis_gates": ["rz","sx","x","cx"],
    "ports": [0,4]
}

G = nx.Graph()
G.add_nodes_from(range(processor_toy["num_qubits"]))
G.add_edges_from(map(tuple, processor_toy["coupling_map"]))
nx.draw(G, with_labels=True, node_size=900)
plt.title("Illustrative 5-qubit processor")
plt.show()
```

7. Bell-State Example

The architecture challenge uses a small Bell-correlation task because it is an interpretable test of quantum coherence.

Preparation:

```
q0: ── H ──●──
           |
q1: ───────X──
```

The challenge brief targets

`|Φ⁺⟩ = (|00⟩ + |11⟩) / √2`

and requires `XX`, `YY`, and `ZZ` correlations. It also warns that Z-basis agreement alone can occur for a classical mixture of `00` and `11`.

```
# Qiskit/Aer example.
# Use the versions/environment supplied by the organizer for the real submission.

from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator

bell = QuantumCircuit(2, 2)
bell.h(0)
bell.cx(0, 1)
bell.measure([0,1], [0,1])

sim = AerSimulator()
result = sim.run(bell, shots=2048).result()
counts = result.get_counts()

print(counts)
bell.draw("mpl")
```

For an ideal Bell pair, the dominant outcomes should be `00` and `11`.

But the real architecture benchmark goes further: it measures `XX`, `YY`, and `ZZ` so that coherence can be tested.

8. Routing and SWAP Overhead

A long-range logical interaction on a line is a useful teaching example.

```
Processor:
0 — 1 — 2 — 3 — 4

Requested logical interaction:
0 ↔ 4
```

The interaction is not directly available.

A compiler can move the logical state through the graph using routing operations.

Conceptually:

```
A B C D E
↓ SWAP
B A C D E
↓ SWAP
B C A D E
↓ SWAP
B C D A E
```

The exact sequence depends on placement, routing method, directionality and target gate set. The important observation is that **distance in the coupling graph can become circuit overhead**.

```python
from qiskit.transpiler import CouplingMap

long_range = QuantumCircuit(5)
long_range.h(0)
long_range.cx(0,4)

compiled = transpile(
    long_range,
    coupling_map=CouplingMap(processor_toy["coupling_map"]),
    optimization_level=0
)

print("Logical circuit:")
print(long_range)
print("\nCompiled circuit:")
print(compiled)
print("\nDepth:", compiled.depth())
print("Operations:", compiled.count_ops())
```

9. Transpilation: Logical Circuit → Physical Circuit

A simplified compilation pipeline:

```
Logical circuit
      ↓
Initial layout / placement
      ↓
Routing
      ↓
Gate decomposition
      ↓
Optimization
      ↓
Native/target circuit
      ↓
Execution
```

For this hackathon, do not treat transpilation as an invisible implementation detail. It is part of the architecture story.

The challenge asks participants to relate changes in fidelity to operations introduced by compilation and to the assumed noise model.

10. Noise and Its Architectural Impact

Quantum operations are imperfect.

A conceptual model is:

```
ideal gate
   ↓
gate error
   ↓
state deviation
   ↓
different measurement statistics
```

Possible modeled effects include:

- one-qubit gate errors,
- two-qubit gate errors,
- readout errors.

For the official common benchmark, use the organizer-defined noise assumptions. Alternative noise models are optional studies and should be separated from the common benchmark.

```
from qiskit_aer.noise import NoiseModel, depolarizing_error, ReadoutError

# Illustrative noise only — replace with official challenge assumptions.
noise_model = NoiseModel()
noise_model.add_all_qubit_quantum_error(
    depolarizing_error(0.001, 1), ["x", "sx"]
)
noise_model.add_all_qubit_quantum_error(
    depolarizing_error(0.01, 2), ["cx"]
)
noise_model.add_all_qubit_readout_error(
    ReadoutError([[0.99,0.01],[0.02,0.98]])
)

print(noise_model)
```

10.1 Why Noise Makes Routing Important

Suppose:

```
Processor A → 4 two-qubit operations
Processor B → 4 algorithmic + 8 routing two-qubit operations
```

Under a noisy model, B has more noisy operations.

But **do not automatically conclude that a more connected processor is universally better**. Gate sets, noise assumptions, finite topology and other architectural properties also matter. The challenge explicitly asks teams to distinguish connectivity effects from gate-set/noise effects.

11. What You Must Measure

11.1 Problem-Specific Metrics

The selected PS determines the primary metric. The official Phase 1 instructions give examples such as:

- energy error,
- objective value,
- accuracy,

- fidelity,
- classification performance,
- benchmark time.

11.2 Architecture / Resource Metrics

Track the metrics applicable to your problem, such as:

- circuit depth,
- two-qubit gate count,
- SWAP count,
- physical qubits,
- ancillas,
- execution cost,
- runtime.

For the Bell-correlation architecture study, the challenge specifically asks for fidelity with uncertainty, depth, two-qubit gates, SWAP overhead, physical qubits and postselection yield.

```
def circuit_metrics(qc):
    ops = qc.count_ops()
    two_q_names = {"cx","cz","ecr","swap","rxx","ryy","rzz"}
    return {
        "depth": qc.depth(),
        "num_qubits": qc.num_qubits,
        "total_ops": sum(ops.values()),
        "two_qubit_ops_estimate": sum(
            n for g,n in ops.items() if g.lower() in two_q_names
        ),
        "swap_count": ops.get("swap", 0),
        "operations": dict(ops)
    }

circuit_metrics(compiled)
```

12. Processor Definition Files

Use a JSON structure that makes the architecture explicit.

```
{
  "name": "processor_example",
  "num_qubits": 5,
  "coupling_map": [[0,1],[1,2],[2,3],[3,4]],
  "basis_gates": ["rz","sx","x","cx"],
  "ports": [0,4],
  "noise": {
    "type": "challenge_defined",
    "source": "official□starter□guide      "
  }
}
```

For the real challenge, replace every illustrative value with the exact organizer-supplied definition.

The Phase 1 **Starter Guide** may contain processor A/B definitions and coupling maps; the exact supplied definitions are what your benchmark must use.

```
from pathlib import Path

def load_processor(path):
    with Path(path).open("r", encoding="utf-8") as f:
        return json.load(f)

# Example:
# processor_A = load_processor("processors/processor_A.json")
# processor_B = load_processor("processors/processor_B.json")
```

13. Select Exactly One Problem Statement

The Phase 1 participant instructions list 20 choices:

13.1 Quantum Chemistry

- C1 — Molecular Ground-State Energy (H_2)
- C2 — Diatomic Potential Energy Surface
- C3 — Hardware-Efficient H_2O
- C-OI — Open Innovation

13.2 Quantum Optimization

- O1 — Max-Cut
- O2 — Portfolio Selection
- O3 — Knapsack
- O-OI — Open Innovation

13.3 Quantum Simulation

- S1 — Ising Chain Ground State
- S2 — Spin Dynamics (Real-Time)
- S3 — Noise in Trotter Simulation
- S-OI — Open Innovation

13.4 Quantum Machine Learning

- Q1 — Biomedical Signal Classifier
- Q2 — Quantum Kernel Classifier
- Q3 — Noise-Resilient Classifier
- Q-OI — Open Innovation

13.5 Post-Quantum Cryptography

- P1 — Quantum-Safe Secure Channel
- P2 — Crypto Inventory & Migration Planner
- P3 — PQC Performance Benchmark
- P-OI — Open Innovation

These names/codes are from the official Phase 1 instructions.

Do not implement multiple PSs for the Phase 1 submission.

```
# Fill these after the team receives/selects its Starter Guide.
SELECTED_PS_CODE = "C1"  # placeholder
SELECTED_PS_NAME = "Molecular Ground-State Energy (H2)"  # placeholder
SELECTED_TRACK = "Quantum Chemistry"  # placeholder

print(SELECTED_PS_CODE, "-", SELECTED_PS_NAME)
print("Track:", SELECTED_TRACK)
```

13.6 PS Specification Sheet

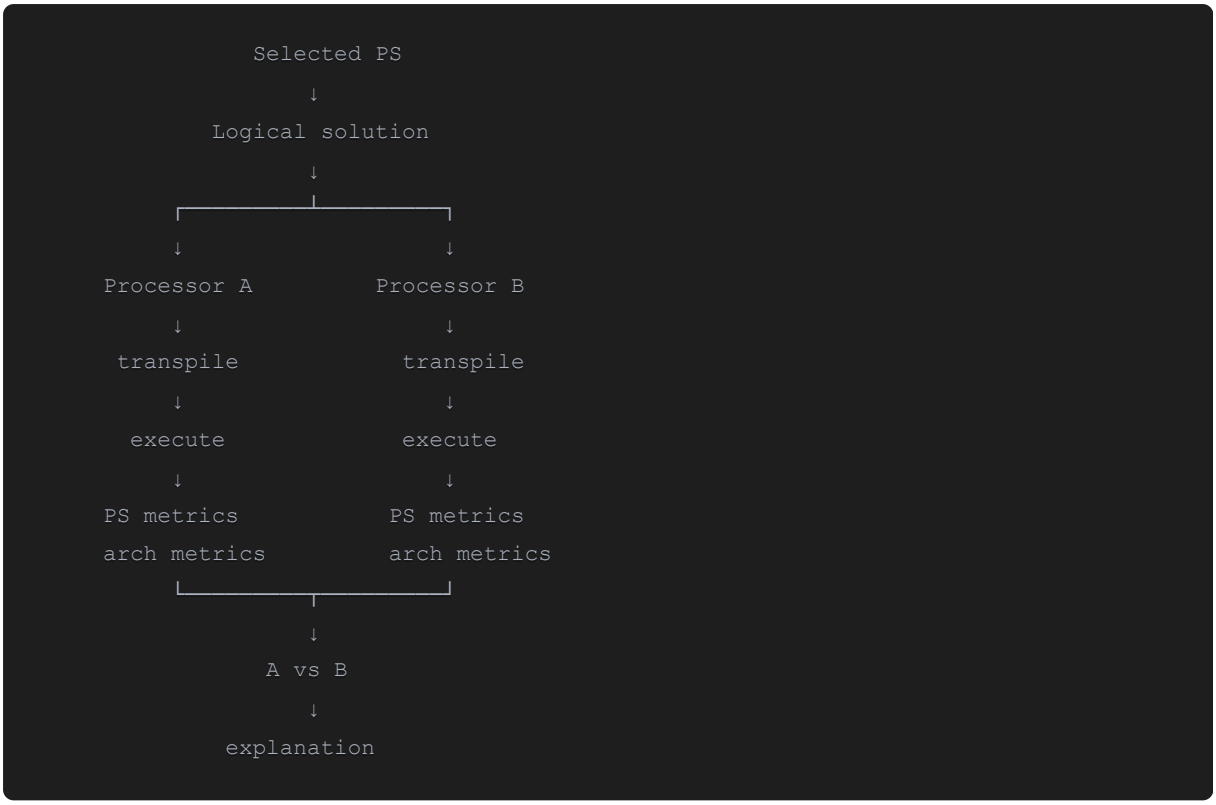
Fill this from the official `Starter Guide`

Item	Your value
PS code	
PS name	
Track	
Input / instance	
Core algorithm	
Required output	
Primary metric	
Reference/classical baseline	
Processor A requirements	
Processor B requirements	
Architecture metrics	

The `Starter Guide` will provide submission templates that define the required Git repository structure, including the files and folders needed for the Phase 1 submission.

14. The Phase-1 A/B Experiment

Use exactly the same problem and controlled settings on both processors.



The Phase 1 instructions require the same solution to be run on A and B and the differences to be explained with measured evidence.

```
def benchmark_circuit(logical_circuit, coupling_map, shots=2048, optimization_level=
1):
    sim = AerSimulator()
    compiled = transpile(
        logical_circuit,
        coupling_map=coupling_map,
        optimization_level=optimization_level
    )
    result = sim.run(compiled, shots=shots).result()
    metrics = circuit_metrics(compiled)
    metrics["shots"] = shots
    metrics["counts"] = result.get_counts()
    return metrics, compiled

# Example:
toy_problem = QuantumCircuit(5)
toy_problem.h(0)
toy_problem.cx(0,4)

metrics_A, compiled_A = benchmark_circuit(
    toy_problem, processor_toy["coupling_map"]
)

metrics_A
```

15. A/B Comparison and Interpretation

Your final table should contain real measured values.

Metric	Processor A	Processor B	Difference
Problem metric			
Depth			
2Q gates			
SWAPs			
Physical qubits			
Runtime			
Fidelity / accuracy / other			

15.1 Interpretation Questions

1. What changed?
2. Which architectural property changed?
3. Did transpilation introduce more routing?
4. How many extra operations appeared?
5. Did the problem metric change?
6. Could gate-set/noise differences also contribute?
7. What remains uncertain?

The official instructions require evidence-based A/B interpretation rather than unsupported claims.

16. Protection and Error-Detection Study

The geometry-aware challenge identifies three directions:

16.1 Parity / Stabilizer Checks

Additional measurements reveal selected errors without directly measuring the encoded information.

16.2 Syndrome Postselection

Keep outcomes that pass a check. Report **both** conditional quality and acceptance/discarded shots.

16.3 Geometry-Aware Encodings

Adapt an encoding/check network to the available connectivity and measure its overhead.

A clean experiment is:

```
Noisy baseline
      ↓
quality + resource measurement
      ↓
add protection
      ↓
quality + resource measurement
      ↓
benefit versus overhead
```

17. Scientific Reporting and Reproducibility

A useful final result is not merely:

> *fidelity = 0.91*

Report enough information to reproduce the observation:

- processor definition,
- circuit version,
- shots,
- seed where applicable,
- noise assumptions,
- depth,
- two-qubit gates,
- routing/SWAPs,
- qubit/ancilla usage,
- postselection yield,
- uncertainty.

For a simple proportion `p` estimated from `N` independent shots:

$\sigma_p \approx \sqrt{p(1-p)/N}$.

Use a more appropriate statistical treatment when the metric requires it.

```python
def proportion_standard_error(p, n):
    return math.sqrt(p * (1-p) / n)

print("Example SE:", proportion_standard_error(0.92, 2048))
```

18. Results, Tables and Figures

```
results/
├── tables/
│   ├── processor_A_metrics.csv
│   ├── processor_B_metrics.csv
│   ├── AB_comparison.csv
│   └── final_comparison.csv
└── figures/
    ├── processor_A_graph.png
    ├── processor_B_graph.png
    ├── logical_vs_transpiled.png
    ├── AB_performance.png
    └── architecture_tradeoffs.png
```

Prefer machine-readable CSV/JSON results over screenshots so that plots can be regenerated.

```python
from pathlib import Path

for folder in [
    Path("results/tables"),
    Path("results/figures")
]:
    folder.mkdir(parents=True, exist_ok=True)

print("Results folders ready.")
```

19. Suggested Approaches by Experience Level

19.1 Explorer

Bell preparation + simple error detection; compare ideal, noisy and protected results.

19.2 Builder

Investigate placement, routing or improved checks; compare fidelity benefit against extra operations.

19.3 Research

Investigate geometry-aware encodings or stabilizer networks; study scaling and noise sensitivity.

These are the experience-level approaches described in the architecture challenge.

20. Phase-1 Workflow Checklist

- [ ] Exactly one PS selected.
- [ ] **Starter Guide** inputs used.
- [ ] Core solution implemented.
- [ ] Processor A run completed.
- [ ] Processor B run completed.
- [ ] A-vs-B comparison completed.
- [ ] Problem metrics recorded.
- [ ] Architecture/resource metrics recorded.
- [ ] Required graphs/tables generated.
- [ ] README completed.
- [ ] Environment specification included.
- [ ] Notebook runs from a clean setup.
- [ ] Repository is accessible.
- [ ] AI assistance disclosed if required.

This follows the organizer's Phase 1 checklist.

21. Final Phase-1 Experiment Structure

Your final notebook should follow this exact story:

```
SELECT ONE PROBLEM STATEMENT
            ↓
Understand the PS + Starter Guide
            ↓
Implement the core solution
            ↓
Build logical circuit / computational workflow
            ↓
Run on Processor A
            ↓
Transpile / execute / measure
            ↓
Run on Processor B
            ↓
Transpile / execute / measure
            ↓
A vs B comparison
            ↓
Explain architecture effects
            ↓
Generate tables + figures
            ↓
Reproducible final repository
```

The official Phase 1 scope ends with the Processor A/B comparison.

22. Phase-1 Submission Checklist

Before submitting, verify:

- [ ] Exactly **one** Problem Statement selected.
- [ ] Problem Statement code and name clearly documented.
- [ ] `Starter Guide` inputs used exactly as specified.
- [ ] Core solution implemented.
- [ ] Processor A run completed.
- [ ] Processor B run completed.
- [ ] Same benchmark conditions used for A and B.
- [ ] Problem-specific performance metrics recorded.
- [ ] Circuit/resource metrics recorded.
- [ ] Transpiled circuits inspected.
- [ ] A-vs-B comparison completed.

- [ ] Architecture differences explained using measured evidence.
- [ ] Required graphs and tables generated.
- [ ] `README.md` completed.
- [ ] `requirements.txt` or equivalent included.
- [ ] Notebook runs from a clean environment.
- [ ] Repository is accessible to judges.
- [ ] AI assistance disclosed if required.

These points follow the organizer's Phase 1 submission and checklist requirements.

23. Participant Lab Notes

Problem Statement

> *What exactly are we solving?*

Method

> *What algorithm/workflow did we implement?*

Processor A

> *What happened after compilation and execution?*

Processor B

> *What happened after compilation and execution?*

A vs B

> *What changed in problem performance, circuit depth, two-qubit gates, routing, qubit usage or runtime?*

Explanation

> *Which processor property explains the observed difference?*

Evidence

> *Which table, figure or measured result supports the explanation?*

Limitations

> *What can this experiment establish, and what remains uncertain?*

23.1 Final Principle

Do not just report the output. Show how the same solution behaves on two different processor architectures and explain the difference using measured evidence.

That is the central Phase 1 objective of this hackathon.